Parcial Algoritmos y Programación II 7541 Curso Calvo (2º instancia) 17 de Junio de 2013

Padrón: **Apellido y Nombre:**

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  int H = 64;
  char G = 'A';

  F = new int[3];

  for (int i = 0; i < 3; i++) {
      F[i] = H + i;
  }

  H++;
  A = F;
  C = new int;
  (*C) = A[1];
  A = F + 2;

  cout << (*F) << (*C) << A[0] << endl;

  B = &C;
  A = C;
  C = F + 1;
  (**B) = H + 2;

  cout <<  (*C) <<  (**B) << (*A) << endl;

  D = (char*) A;
  E = (char*) *B;
  F[1] = 68;

  cout << (*D) << (*A) << (*E) << endl;

  E = &G;
  H++;
  G = (char) H;
  H++;
  (*A) = H;

  cout << (*E) << G << (*D) << endl;


  // liberar la memoria
  // ...

  return 0;
}
```

2. Implementar para la clase ListaDoblementeEnlazada<T> la siguiente operación:

```
/*
 * post: invierte el orden de todos los elementos en la Lista: 1º → Nº, 2º → (N-1)º, 3º → (N-2)º, etc.
 */
void invertir();
```

3. Implementar el método buscarConcursosIrregulares de la clase AuditorDeConcursos a partir de las siguientes especificaciones:

```
class AuditorDeConcursos {
  public:
    /* post: busca en concursosAuditados aquellos Concursos que tengan entre sus premiados algún nombre que no
     *       corresponda con el de un Participante asociado al Concurso.
     *       Devuelve una nueva Lista con todos los Concursos en esta condición. */
    Lista<Concurso*>* buscarConcursosIrregulares(Lista<Concurso*>* concursosAuditados);
};
```

```
class Concurso {
  public:
    /* post: inicializa el consurso sin participantes
     *       asociados, con el nombre indicado.
     */
    Concurso(string nombre);
    /* post: destruye todos los participantes asociados.
     */
    ~Concurso();
    /* post: devuelve el nombre que identifica el consurso.
     */
    string getNombre();
    /* post: devuelve los participantes asociados
     */
    Lista<Participante*>* getParticipante();
    /* post: devuelve los nombres de aquellos participantes
     *       que han sido premiados por su participación.
     */
    Lista<string>* getPremiados();
};
```

```
class Participante {

  public:
    /* post: inicializa el participantes con el nombre
indicado
     *       y 0 puntos.
     */
    Participante(string nombre);
    /* post: devuelve el nombre que los identifica
     */
    string getNombre();
    /* post: devuelve la cantidad de puntos del participante
     */
    int getPuntos();
    /* post: cambia por cantidad los puntos del participante
     */
    void setPuntos(int cantidad);
};
```

4. Diseñar la especificación e implementar el TDA **MarcadorDeBasquet**. Un Marcador de Basquet se crea indicando los nombres de los equipos que compiten: local y visitante. Un Marcador puede estar: no iniciado, en el primer cuarto, en el segundo cuarto, en el tercer cuarto, en el cuarto cuarto o terminado. Debe proveer operaciones para:

- avanzar al siguiente estado
- devolver el estado en el que se encuentra
- anotar un tiro libre, doble o triple al equipo local o al visitante
- devolver la cantidad total de puntos del local o del visitante
- devolver la cantidad de puntos en un cuarto del local o del visitante
- devolver la máxima distancia en puntos alcanzada entre los dos equipos

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas